

Agentic Python Verifier: Proof-Driven Python Code Generation via PlusCal Invariant Co-Synthesis

Harry Wang Eric Shang Nan Chen

Final Report
May 28, 2026

Abstract

This project implements a prototype proof-driven Python generation pipeline. Given a natural-language requirement, a synthesis agent proposes a PlusCal algorithm, a candidate inductive invariant, a safety property, and finite constant domains. A verifier translates the PlusCal module to TLA+ and checks three obligations with TLC: initiation, consecution, and invariant implication of the target property. When a check fails, the system parses TLC output into a structured counterexample and sends that feedback to the synthesis agent for a repair attempt. Only after all obligations pass does a refinement agent emit a Python module with runtime assertions derived from the invariant. The final repository contains the core pipeline, command-line interface, Anthropic tool-call boundaries, TLC configuration generation, counterexample parsing, example prompts, offline unit tests, mocked pipeline tests, and gated integration tests for TLC and live end-to-end execution.

1 Project Overview

The goal of the Agentic Python Verifier is to invert the usual order of large-language-model code generation. Instead of generating Python first and then attempting to test or verify it afterward, the repository first builds a formal artifact: a PlusCal algorithm with an explicit inductive invariant and safety property. The verifier checks the formal artifact before the system is allowed to refine it into executable Python.

The current implementation is organized around `src/main.py`. The pipeline accepts a `TaskRequest`, constructs runtime settings, creates artifact directories, asks the synthesis agent for an initial proposal, checks the proposal with TLC, repairs failed proposals up to an iteration cap, and then invokes the refinement agent only when the formal proof bundle passes. The pipeline returns a structured `PipelineResult` with either `verified` or `unverified` status.

The final repository includes:

- synthesis, repair, verification, and refinement agents;
- a formal layer for PlusCal translation, TLC invocation, TLC configuration generation, and TLC output parsing;
- Pydantic models for task requests, synthesis proposals, proof bundles, counterexamples, and pipeline results;

- a Typer command-line interface;
- example benchmark prompts for a bounded counter, bank transfer, resource semaphore, and ring buffer;
- offline tests that mock Anthropic and use captured TLC outputs; and
- gated integration tests for real TLC and live end-to-end execution.

2 Motivation

LLM-generated code is often plausible but difficult to trust. Unit tests can catch many defects, but they sample behavior chosen by the developer or model. For stateful algorithms, the most important requirements are often safety properties such as “the counter never exceeds the maximum,” “balances never become negative,” or “the total amount of money is preserved.” These requirements naturally fit model checking, where all states in a finite model can be explored rather than only selected examples.

The project therefore uses a formal intermediate representation as the trust boundary. PlusCal gives an algorithmic notation that can be translated to TLA+, and TLC can exhaustively check finite-state models [1, 2, 3]. The language model still contributes creativity: it proposes the algorithm, invariant, property, and repairs. However, the verifier decides whether the proposal is acceptable. This creates a separation of concerns: the model proposes, TLC checks, and Python is emitted only after the formal artifact survives the proof obligations.

3 Architecture

The repository follows a staged architecture. The most important modules are shown in Table 1.

Module	Responsibility
<code>main.py</code>	Orchestrates the propose-check-repair-refine loop and writes verified artifacts.
<code>synth_agent.py</code>	Calls the LLM tool for initial PlusCal and invariant proposals, then calls the repair tool after failures.
<code>verifier.py</code> <code>refine_agent.py</code>	Runs PlusCal translation and all three TLC proof obligations. Sends the verified module and invariant conjuncts to the Python refinement tool.
<code>tlc_config.py</code>	Generates TLC configuration files and auxiliary TLA+ modules for consecution and property checking.
<code>counterexample_parser.py</code>	Converts raw TLC output into structured pass/fail/error/timeout results and counterexample traces.
<code>tools.py</code>	Defines the Anthropic tool schemas for proposal, repair, and Python emission.

Table 1: Primary implementation components.

The complete dataflow is:

1. The user supplies a natural-language requirement.

2. `SynthesisAgent.propose` requests a structured tool call containing a TLA+/PlusCal module, invariant name, property name, and finite constant domains.
3. `Verifier.check` writes the proposal into a per-iteration work directory, invokes `pcal.trans`, writes auxiliary modules and `.cfg` files, and invokes TLC for each obligation.
4. If any obligation fails, `SynthesisAgent.repair` serializes the failing proof bundle into an Anthropic `tool_result` message.
5. If all obligations pass, `RefineAgent.to_python` requests a Python module and assertion map.
6. The final TLA+ and Python files are written under the configured generated artifact directory.

This architecture is intentionally narrow. The verifier has no LLM logic, the LLM agents do not decide proof success, and the models make the boundary data explicit.

4 Formal Verification Approach

For a generated algorithm, the verifier expects the translated TLA+ module to define an initial predicate `Init`, transition relation `Next`, state tuple `vars`, candidate invariant `Inv`, and target property `Property`. The verifier checks the three obligations in Table 2.

Obligation	Formula	TLC Encoding
Initiation / reachable invariant check	$Init \Rightarrow Inv$ over the generated <code>Spec</code> run	Run TLC on the translated module with <code>SPECIFICATION Spec</code> and <code>INVARIANT Inv</code> . This also checks reachable preservation under <code>Spec</code> ; a later reachable transition violation can appear in this run.
Consecution	$Inv \wedge Next \Rightarrow Inv'$	Generate <code>Consec_<Module>.tla</code> with <code>IInit == Inv</code> and an <code>ISpec</code> formula that starts from <code>IInit</code> and takes <code>Next</code> steps over <code>vars</code> ; check <code>Inv</code> .
Property implication	$Inv \Rightarrow Property$	Generate <code>Prop_<Module>.tla</code> with <code>PInit == Inv</code> and an unchanged transition; check <code>Property</code> .

Table 2: Implemented proof obligations.

The key design decision is to use `Inv` as an initial-state predicate for the consecution and property checks. This makes TLC enumerate all states that satisfy the candidate invariant and then verify either one-step preservation or property satisfaction. The prompt therefore instructs the synthesis model to make every variable enumerable with explicit finite set membership, such as:

```
Inv == /\ counter \in 0..MaxValue
      /\ pc \in {"Increment", "Done"}

Property == counter <= MaxValue
```

Without those finite membership constraints, TLC may be unable to enumerate the invariant state space, or the search may be unbounded. The implementation handles this by surfacing TLC evaluation errors and timeouts as structured repair feedback rather than treating them as successful checks.

5 Counterexample-Guided Repair Loop

The repair loop is the main agentic component of the system. The synthesis agent is not limited to one-shot generation. When a proposal fails, the verifier returns a **ProofBundle** containing one result for initiation, one for consecution, and one for property implication. Each **ObligationResult** can represent **passed**, **failed**, **timeout**, or **error**.

For invariant and initial-state violations, the parser extracts the violated predicate and a trace of TLC states. A consecution failure can therefore become repair feedback in this shape:

```
- consec: FAILED
  violated_predicate: Inv
  State 1 <Initial predicate>: counter=11
```

The repair agent receives this feedback together with the previous proposal metadata. Its tool schema requires a revised module, the targeted obligation, and a short diagnosis. This makes repair attempts inspectable: a later user or test can see whether the model believed it was fixing an initiation, consecution, or property implication failure.

The loop stops in one of two states. If all obligations pass before the iteration cap, the system refines to Python. If the cap is exhausted, the pipeline returns **unverified** and does not emit a Python module from the failed proposal.

6 Python Refinement

Python generation is intentionally downstream of verification. The refinement agent receives the verified TLA+/PlusCal module and a best-effort split of the top-level invariant conjuncts. Its system prompt requires:

- a complete Python 3.11+ module using only the standard library;
- runtime assertions at the entry and exit of every state-mutating function;
- a module docstring containing the PlusCal module name, invariant, and property; and
- an `assertion_map` connecting each TLA+ invariant conjunct to a Python check.

For example, an invariant over a bounded counter can be transformed into Python checks around the mutating operation:

```
def increment(counter: int, max_value: int = 10) -> int:
    assert 0 <= counter <= max_value
    if counter < max_value:
        counter += 1
    assert 0 <= counter <= max_value
    return counter
```

These assertions are not a substitute for proving the Python implementation equivalent to the PlusCal algorithm. They are a refinement guard: the generated code carries checks derived from the verified invariant, and the `assertion_map` provides traceability from the formal artifact to the runtime code.

7 Benchmarks and Examples

The checked-in example prompts are intentionally small because TLC performs finite-state exploration and because the main research question is whether the pipeline structure works. The repository contains four example requirements:

Implement a bounded counter from 0 to 10.

Implement a bank transfer between two accounts that never allows a negative balance and preserves the total sum of money.

Implement a finite counting semaphore with Capacity = 2 and at most three clients.

Implement a finite ring buffer with Capacity = 3 over a small item set.

The test fixtures also include two bounded-counter TLA+ modules. One fixture uses a suitable invariant for a counter bounded by `MaxValue`; the other intentionally makes the invariant too strong by excluding the final allowed counter value. The gated TLC integration test expects the good fixture to pass all three obligations and the bad fixture to fail initiation or consecution with a counterexample.

The current repository does not include saved successful live Anthropic conversation logs or generated final Python artifacts from a real end-to-end run. Therefore, this report treats the benchmarks as prompts and test fixtures, not as evidence of completed live benchmark runs.

8 Testing and Evaluation

The project has a broad offline test suite. The tests avoid requiring an API key or TLA+ tools jar by mocking the Anthropic boundary and using captured TLC trace fixtures. The main tested areas are:

- model validation for task, synthesis, repair, and proof objects;
- runtime configuration and fail-fast error handling;
- CLI help text and missing-configuration behavior;
- TLC configuration and auxiliary module generation;
- TLC output parsing for passed runs, invariant violations, initial predicate violations, parse errors, and timeouts;
- Anthropic client overload fallback behavior;
- mocked synthesis and repair tool-call handling;

- mocked refinement and invariant-conjunct extraction; and
- mocked end-to-end pipeline behavior for first-try success, repair then success, and iteration exhaustion.

The repository also defines integration tests, but they are gated:

- `tests/test_tlc_runner.py` requires `TLA2TOOLS_JAR` or `TLA_TLC_JAR`. It runs the verifier against known good and bad TLA+ fixtures.
- `tests/test_e2e.py` requires both `ANTHROPIC_API_KEY` and a TLA+ tools jar. It performs a live bounded-counter pipeline run and may incur API costs.

This distinction is important for interpreting the final state. The checked-in test suite demonstrates that the pipeline wiring, parser, configuration generation, and mocked agent loop behave as intended. It does not, by itself, prove that live Anthropic calls are deterministic or that the model reliably solves every benchmark prompt. The code is designed so those live runs can be performed, but this report does not claim live evaluation results that are not present in the repository.

9 Limitations

The most important limitation is that verification is finite-domain model checking, not unbounded theorem proving. The system asks the model to provide small finite constant domains so TLC can explore the state space. This is appropriate for a prototype but weaker than a general proof in TLAPS or another theorem prover.

Second, the invariant must be enumerable as an initial-state predicate. This is a practical constraint introduced by the consecution and property encodings. An invariant that is logically reasonable but does not bound all variables with finite membership constraints may cause TLC errors or timeouts.

Third, refinement is not formally verified. The PlusCal artifact can pass TLC, and the generated Python can include invariant-derived assertions, but the repository does not prove semantic equivalence between the PlusCal algorithm and the Python implementation. The offline suite now includes deterministic Python-level checks for a mocked generated module, but broader generated-code testing remains future work.

Fourth, live LLM behavior is non-deterministic. The repository mitigates this through structured tool schemas, prompt constraints, and offline mocked tests, but repeatability ultimately depends on the checked TLA+ artifact, not on the language-model transcript.

Finally, the invariant splitter in `RefineAgent` is a best-effort top-level / scanner. It avoids splitting nested parenthesized, bracketed, or braced expressions, but more complex TLA+ syntax may require a syntax-aware parser to preserve structure reliably.

10 Future Work

The most immediate next step is to run and archive representative live end-to-end evaluations with a real Anthropic API key and `tla2tools.jar`. Those artifacts should include the prompt, final TLA+ module, TLC outputs, repair iteration logs, and generated Python module.

Additional improvements include:

- expanding the benchmark suite beyond the current four prompt files to schedulers, locks, retry policies, and distributed protocol fragments;
- broadening Python-level tests for generated modules so more runtime assertions and functional behaviors are exercised after refinement;
- replacing the invariant-conjunct splitter with a TLA+-aware parser;
- enriching the current JSON/Markdown repair reports with links to TLC output excerpts and side-by-side proposal diffs;
- adding stronger validation of generated PlusCal before invoking TLC; and
- exploring a theorem-prover backend for unbounded proof obligations.

11 Conclusion

The final repository implements the core of a proof-driven Python generation system. It can request a PlusCal/invariant/property proposal, translate it, check initiation, consecution, and property implication with TLC, parse counterexamples into structured repair feedback, and refine verified proposals into Python with invariant-derived assertions. The offline test suite validates the main internal contracts and the mocked agent loop, while gated integration tests provide a path to real TLC and live end-to-end evaluation.

The project should be understood as a working prototype rather than a complete verification platform. Its strongest contribution is the architecture: the LLM is used for synthesis and repair, while TLC is responsible for accepting or rejecting formal artifacts before Python code is produced.

References

- [1] Leslie Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.
- [2] Leslie Lamport. The pluscal algorithm language. *Theoretical Aspects of Computing - ICTAC 2009*, pages 36–60, 2009.
- [3] Yuan Yu, Panagiotis Manolios, and Leslie Lamport. Model checking tla+ specifications. In *Correct Hardware Design and Verification Methods*, pages 54–66. Springer, 1999.